Resolución de un problema mediante búsqueda heurística

Actividad grupal · Algoritmo A* aplicado al problema del Robot Amazon

Asignatura: Razonamiento y planificación automática — Primer Semestre

Universidad: Universidad Internacional de La Rioja (UNIR) — Máster Universitario en Inteligencia Artificial

Equipo 1073F:

- Adonai Samael Hernandez Mata
- Diego Alfonso Najera Ortiz
- Mauricio Alberto Alvares Aspeitia
- Cesar Ivan Martinez Perez

Fecha: Mayo 2026

---

1. Análisis del problema

El robot de Amazon arranca en `(2,2)` dentro de una matriz 4×4 con dos paredes fijas en `(0,1)` y `(1,1)`, y debe reubicar tres inventarios. La tabla resume el origen y destino de cada uno:

Inventario	Inicial	Objetivo
M1	(0,0)	(3,3)
M2	(2,0)	(3,2)
M3	(0,3)	(3,1)

Las acciones disponibles son los cuatro movimientos cardinales más cargar y descargar inventario, todas con coste `g = 1`. La heurística obligatoria es la distancia de Manhattan.

2. Decisiones de modelado

2.1 Hallazgo del equipo: la carga es necesariamente adyacente

Al analizar las acciones del robot identificamos una ambigüedad: el enunciado describe `cargar R M{i} fila{f} columna{c}` sin precisar si la acción se ejecuta **estando sobre la celda del inventario** o **desde una celda contigua**. Repasando todas las restricciones de movimiento del documento, encontramos que la regla *"el robot no puede entrar a una casilla ocupada por un inventario que no esté cargando"* (sección 8 de la

especificación) cierra el caso: si el robot no puede pisar la casilla del inventario, no es posible cargarlo desde encima. La única lectura compatible con esa regla es que **cargar y descargar se hacen desde una celda adyacente** y el robot nunca pisa la celda del inventario.

Este hallazgo tiene tres consecuencias prácticas:

- Cada operación de carga o descarga **añade exactamente 1 al coste real** sobre la ruta directa de Manhattan, lo que refina la heurística (ver §3).
- El factor de ramificación del espacio de estados aumenta respecto a un modelo "carga sobre celda", porque el robot debe rodear cada inventario.
- Permite reportar la dinámica completa del problema (ruta más corta más ajustes de adyacencia), no sólo la trayectoria directa.

2.2 Otras decisiones de modelado

Además del hallazgo anterior, fijamos tres convenciones más para mantener consistencia con el enunciado:

- **El inventario viaja con el robot** mientras está cargado y la celda origen queda libre, lo que evita estados inconsistentes y respeta la intuición operativa de un almacén real.
- **El robot transporta un solo inventario a la vez** y su posición final

no está restringida — lo único que importa es dónde quedan `M1`, `M2` y `M3`.

- **La notación de acciones replica al enunciado al pie de la letra:** `mover R fila{f} columna{c}`, `cargar R M{i} fila{f} columna{c}` y `descargar R M{i} fila{f} columna{c}`.

A nivel de código, el estado se implementó como `@dataclass(frozen=True, slots=True)` para que sea inmutable y hasheable (el robot, qué carga y las tres posiciones de inventario); el nodo del árbol agrega padre, acción, `g` y `h`; la frontera es un `heapq` ordenado por `(f, h, contador FIFO)`; y la lista cerrada es un `dict[Estado, int]` con el mejor `g` conocido por estado.

3. Heurística

`h(n) = Σ manhattan(Mi, obj_Mi) + max(0, manhattan(R, M_más_cercano)-1)` cuando el robot no carga nada (-1 porque cargar es desde celda **adyacente**).

Es admisible porque ignora paredes, no penaliza las acciones de carga y descarga (cada una suma 1 al coste real) y no fuerza ningún orden entre inventarios — todos esos factores sólo pueden aumentar el coste real, así que la suma siempre es una cota inferior. Para el estado inicial: `6 + 3 + 5 + 1 = 15`, lo que da `f(inicial) = g + h = 0 + 15 = 15`.

4. Algoritmo A*

La implementación es la versión estándar del libro de Russell y Norvig, con dos detalles que vale la pena destacar. Primero, el desempate: cuando dos nodos tienen el mismo `f`, se prefiere el de menor `h`, y si vuelven a empatar, el más antiguo (FIFO) — esto vuelve la traza reproducible entre corridas. Segundo, la lista cerrada se guarda como `dict[Estado, int]` recordando el mejor `g` visto; un estado puede reabrirse sólo si reaparece con un `g` estrictamente menor, lo que en la práctica casi no ocurre porque nuestra heurística es consistente.

```
abierta.push(Nodo(s0, g=0, h=h(s0)))
while not abierta.vacia():
    n = abierta.pop()                        # menor f, luego h, luego FIFO
    if n.estado in cerrada and cerrada[n.estado] <= n.g: continue
    cerrada[n.estado] = n.g
    if es_objetivo(n.estado): return reconstruir_camino(n)
    for accion, suc in sucesores(n.estado):
        g2 = n.g + 1
        if suc in cerrada and cerrada[suc] <= g2: continue
        abierta.push(Nodo(suc, n, accion, g2, h(suc)))
```

5. Plan de desarrollo y pruebas

El trabajo se ordenó en seis tramos: fijar las suposiciones de modelado, implementar `estado.py`/`acciones.py`/`heuristica.py`, después la infraestructura del árbol (`nodo.py`, `frontera.py`, `astar.py`), seguido por `traza.py` y `main.py` para cubrir los criterios 2 y 3 de la rúbrica,

y finalmente la batería de tests y la corrida que alimenta esta memoria.

La suite tiene **24 pruebas** ejecutables con `python -m unittest discover`

(Ran 24 tests in 0.041s — OK):

Archivo	Qué verifica
`test_heuristica.py`	Manhattan en valores conocidos, `h(inicial)=15`, admisibilidad.
`test_sucesores.py`	Movimientos válidos, no pisa paredes ni inventarios, carga adyacente.
`test_astar.py`	Caso trivial, problema completo, coste óptimo = **19 acciones**.
`test_secuencia.py`	Simulación inversa al estado objetivo; notación correcta; 3 cargas / 3 descargas.

6. Resultado de la corrida final

```
Iteraciones (nodos expandidos): 323
Nodos generados:                836
Coste total g(objetivo):        19
Longitud de la secuencia:       19
```

Las 323 corresponden a **expansiones** del bucle principal de A*, no a la longitud del plan: A* abre y descarta cientos de nodos antes de estabilizar la ruta óptima, que termina siendo de **19 acciones**. La distinción es relevante porque algunas implementaciones reportan

únicamente los pasos del camino devuelto y eso da la impresión de que el algoritmo apenas trabajó.

Secuencia óptima (notación del enunciado):

```
 1. mover R fila2 columna1
 2. cargar R M2 fila2 columna0
 3. mover R fila2 columna2
 4. descargar R M2 fila3 columna2
 5. mover R fila2 columna1
 6. mover R fila2 columna0
 7. mover R fila1 columna0
 8. cargar R M1 fila0 columna0
 9. mover R fila2 columna0
10. mover R fila2 columna1
11. mover R fila2 columna2
12. mover R fila2 columna3
13. descargar R M1 fila3 columna3
14. mover R fila1 columna3
15. cargar R M3 fila0 columna3
16. mover R fila2 columna3
17. mover R fila2 columna2
18. mover R fila2 columna1
19. descargar R M3 fila3 columna1
```

En la traza, durante el descenso óptimo `f` se mantiene en 15: cada paso suma 1 a `g` y resta 1 a `h`, lo que confirma que la heurística está bien calibrada en la zona crítica. La traza por iteración con listas abierta y cerrada se incluye como anexo en `salidas/traza_corrida_final.txt` (versión resumida) y `salidas/traza_completa.txt` (las 323 expansiones detalladas).

7. Capturas y dificultades encontradas

Para verificar visualmente el comportamiento del solver, montamos una herramienta interactiva sobre el Forest Design System (Tailwind 4). La Figura 1 muestra la pantalla con los KPIs y la secuencia óptima — los mismos 19 pasos del listado anterior. La Figura 2 reproduce la vista "Iteraciones" con `g`, `h`, `f` por nodo y el tamaño de cada lista,

material útil para el criterio 3 de la rúbrica.

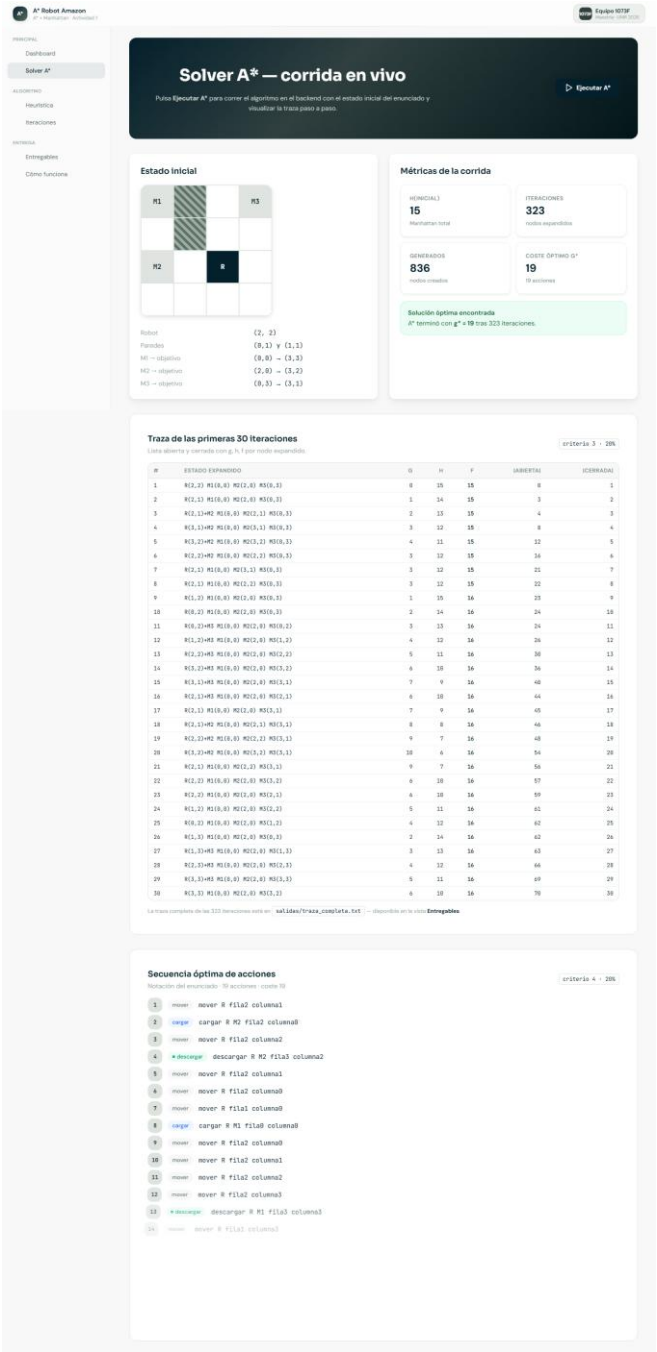

Figura 1. Resultado de A — KPIs y secuencia óptima de 19 acciones.*

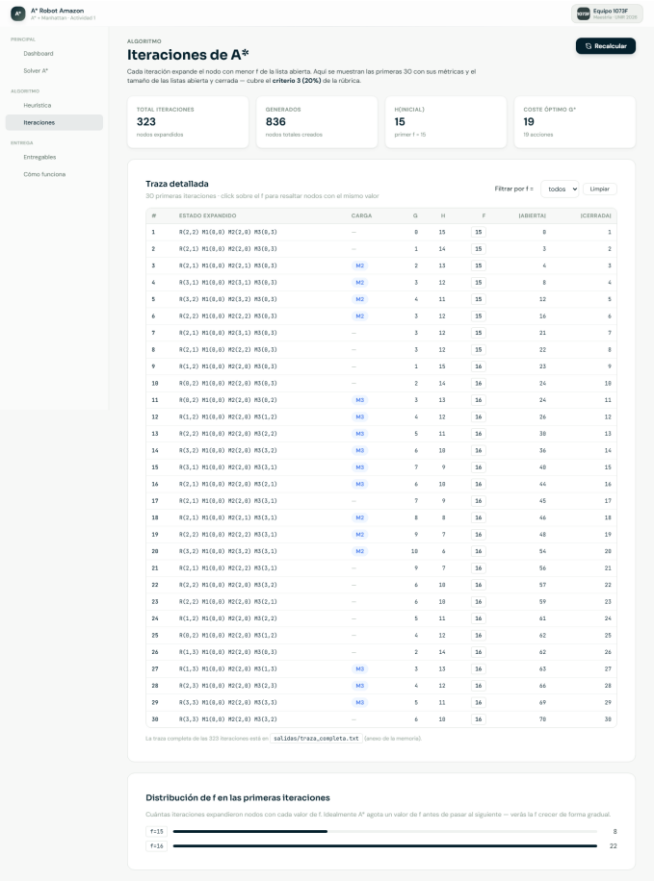

Figura 2. Vista "Iteraciones" — traza con `g`, `h`, `f` y tamaños de listas abierta/cerrada.

Durante la implementación encontramos dos dificultades técnicas. La primera tuvo que ver con el volumen de la traza: las 323 expansiones, imprimiendo la lista cerrada completa, producen alrededor de 139 mil líneas; para evitar archivos inmanejables añadimos un modo "primeras N detalladas + resto resumido" y dejamos la traza completa sólo como anexo. La segunda fue el desempate entre nodos con el mismo `f`: sin un criterio explícito el orden cambiaba entre corridas y la traza dejaba de ser reproducible, problema que se atacó fijando la tupla

`(f, h, contador FIFO)` dentro del `heapq`. A esto se suma el hallazgo sobre la adyacencia de carga que ya discutimos en la sección 2.1, que también nos hizo replantear la heurística inicial.

8. Cómo ejecutar

Desde el directorio `codigo/` del paquete entregado, con Python 3.10+:

```
python main.py                    # corrida con traza compacta
python main.py --completa         # las 323 iteraciones detalladas
python -m unittest discover -s tests   # 24 tests
```

9. Referencias bibliográficas (APA)

- Russell, S. J., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
- Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107. https://doi.org/10.1109/TSSC.1968.300136
- Pearl, J. (1984). *Heuristics: Intelligent search strategies for computer problem solving*. Addison-Wesley.
- Python Software Foundation. (2025). *Python 3.13 documentation — heapq, dataclasses*. https://docs.python.org/3.13/

Anexo A — Estructura del código fuente

```
astar_robot_amazon/
├── backend/
│   ├── app/
│   │   └── solver/
│   │       ├── estado.py          # Estado inmutable, paredes, objetivo
│   │       ├── acciones.py        # TipoAccion + sucesores válidos
│   │       ├── heuristica.py      # Manhattan + h(n) admisible
│   │       ├── nodo.py            # Nodo del árbol + reconstrucción del
camino
│   │       ├── frontera.py        # Cola de prioridad con desempate
explícito
│   │       ├── traza.py           # Render del tablero y salida por
iteración
│   │       ├── astar.py           # Algoritmo principal
│   │       └── main.py            # Entry point con argumentos CLI
│   └── tests/                     # 24 tests unitarios
├── salidas/
│   ├── traza_corrida_final.txt    # Salida resumida (anexo principal)
│   └── traza_completa.txt         # Todas las iteraciones detalladas
└── memoria/
    └── memoria.md                 # Este documento
```

Anexo B — Código fuente

*(Se adjuntan los archivos del directorio `backend/app/solver/` y

`backend/tests/`. No cuentan para el límite de páginas del cuerpo.)*